Chapter 17

Alternative Data and Deep Learning in Finance

17.1 Introduction

Chapter 1's table of financial data set aside one row, alternative data, for this chapter: non-traditional sources such as satellite imagery, web traffic, or shipping data that proxy for economic activity. Chapter 3 noted that large asset managers already parse filings and calls faster than human analysts, and Chapter 4 that alternative data can supplement the delayed, discretionary numbers in financial statements. This chapter turns to data sources that did not exist, in usable form, a generation ago, and to the deep learning architectures built to extract signal from them. It extends Chapter 6's feedforward network and Chapter 16's transformer architecture to two more data shapes, grid-structured images and ordered sequences, and works through the full pipeline from raw data to a usable signal, treating the biases specific to this data as a first-class topic.

Thirteen worked examples run through the chapter. The first five follow a signal from data to trade: a vendor subscription's cost against the trading edge it buys; a regression of a retailer's revenue growth on a satellite-derived car-count proxy; a multi-source earnings-surprise signal, evaluated as Chapter 13 evaluates any strategy; the same signal rebuilt to respect real data-delivery lag; and a peer-relative normalization of the satellite proxy. Three hand-computed networks follow: a two-dimensional convolution over a satellite image patch, a recurrent neural network forward pass over web-traffic readings, and an autoencoder that flags a data-quality anomaly. A Loughran-McDonald tone score built from a company's own risk-factor disclosures shows the appeal and the limits of an in-house ESG-style signal. The last four go deeper: a transfer-learning fine-tuning step, a graph neural network update over a shell-company network, a cross-attention fusion of three data modalities, and a contrastive-pretraining calculation that learns without labels.

The chapter follows the arc of a real alternative-data project. It begins with what alternative data is and how a fund acquires and evaluates it (Sections 17.2.1 and 17.2.2), then turns to the analytics: a single-source regression, a multi-source signal, the point-in-time discipline that keeps that signal honest, and the seasonal and peer-relative adjustments that make raw readings usable (Sections 17.2.3 through 17.3.2).

It then covers the deep learning architectures that extract features from raw images and sequences (Section 17.4), the data-quality and governance problems specific to this data (Section 17.3.3), an in-house ESG-style text signal as a partial, auditable response to those problems (Section 17.5.1), and the more advanced architectures where the field is moving: transfer learning, graph neural networks, cross-attention fusion, and contrastive pretraining (Section 17.5.2). A case vignette and a closing set of challenges tie these techniques to the economic and regulatory stakes of using data that, in most cases, did not exist in usable form before this decade.

17.2 The Alternative-Data Landscape

This part establishes what alternative data is, who sells it, and works two signals end to end.

Wulin.Suo@Queensu.ca

17.2.1 What Counts as Alternative Data

Suppose a hedge fund wants to know how a retailer's holiday-quarter sales are trending three months before the retailer reports earnings. The information is valuable if the fund can act on it early and worthless once everyone else has it. Traditional market, fundamental, and macroeconomic data (Chapter 1's Table 1.6) cannot answer this: market data reflects only what the market already knows, and fundamental data about this quarter will not exist until the retailer files it.

Alternative data exists to fill this gap. In industry usage, it means any data source used to inform an investment or credit decision that falls outside the traditional trio of market, fundamental, and macroeconomic data. The term is a moving target: yesterday's alternative data becomes today's mainstream input once enough participants adopt it, so the boundary is set by current practice rather than by any technical property. Table 17.1 catalogs the categories most widely used.

Category	Examples	Typical financial use
Satellite and aerial imagery	Parking-lot car counts, crop health indices, oil-storage tank levels, construction progress	Retail sales nowcasting, commodity supply estimation, real-estate tracking
Web and app data	Website traffic, app downloads and reviews, job postings, e-commerce pricing	Revenue nowcasting, hiring-trend and competitive signals
Transaction and card-panel data	Aggregated, anonymized consumer card-spending panels	Same-store sales nowcasting ahead of earnings
Geolocation and foot traffic	Mobile-device location pings at store or venue locations	Foot-traffic proxies for retail, dining, and travel
Shipping and logistics	Vessel-tracking (AIS) data, port congestion, rail and truck telemetry	Commodity flow and trade-volume forecasting
Social and search data	Social-media posts, review platforms, search-trend indices	Consumer-sentiment and brand-momentum signals
On-chain (blockchain) data	Public transaction ledgers, wallet flows, exchange inflows and outflows, stablecoin issuance	Crypto asset-flow signals, exchange-risk monitoring, AML and fraud forensics

Table 17.1: Major categories of alternative data

Two properties distinguish alternative data from the data earlier chapters used. First, it usually arrives already processed by a vendor (a satellite imagery provider sells car counts, not raw pixels), so the modeler inherits whatever assumptions and errors are baked into that proprietary step, typically without seeing it. Second, it comes at a frequency and granularity (individual stores, individual weeks) that quarterly, firm-level fundamental data does not offer. That is why it is valuable: it lets an analyst estimate a firm's performance before the firm reports it.

The table's last category inverts both properties. **On-chain data**, the complete public transaction ledger a blockchain such as Bitcoin's or Ethereum's maintains by construction, records every transfer, wallet balance, and smart-contract interaction, free of charge and with no vendor between the analyst and the raw record. Funds trading crypto assets (Chapter 13's tokenization discussion introduces the instruments) read signals from it: net flows into and out of exchange-controlled wallets as a proxy for imminent selling or accumulation, and stablecoin issuance as a proxy for capital entering the ecosystem. Risk and compliance teams watch the same ledger to monitor exchange solvency and trace stolen or laundered funds. What the analyst inherits instead of a vendor's opaque preprocessing is pseudonymity: every party is an unlabeled address, so the commercial value lies in entity labeling and address clustering, the graph techniques Chapter 15 applies to shell-company layering chains. On-chain data is thus the rare category where the raw data is free and complete and the interpretive layer is the scarce, priced asset, the reverse of a satellite feed, whose pixels are expensive and whose car counts are the deliverable.

Wulin.Suo@Queensu.ca

17.2.2 The Alternative-Data Vendor Ecosystem

Unlike the market and fundamental data of Chapters 2 through 5, which arrive through established exchange feeds and regulatory filings, alternative data is bought in a young and heterogeneous vendor market, and how a fund evaluates and integrates a vendor matters almost as much as the data itself. Procurement typically runs through three stages.

First, the fund identifies a hypothesis (does satellite-derived parking-lot data predict Cascade's revenue?) and requests a trial dataset covering a historical period the vendor already has on file. Second, it backtests the trial data against already-reported outcomes, the exercise Section 17.2.3 works by hand, before paying for a live subscription. A vendor unwilling to provide a meaningful historical trial, or whose trial performance does not hold up when checked, is a warning sign. Third, if the trial convinces, the fund negotiates a license that specifies not just price but exclusivity (who else subscribes, and how many), delivery latency (Section 17.3.3 explains why this matters), and historical backfill (how far back the data goes, which bounds the fund's own model validation).

This procurement cycle is itself a source of adverse selection. A vendor has every incentive to showcase the period and the firms where its data performed best, since that is what sells subscriptions, so strong trial performance is not evidence that the data will perform as well later or on other firms. This is Chapter 6's multiple-testing concern about a researcher's model search, applied to a vendor's marketing.

A subscription decision comes down to whether the expected trading edge, scaled to the capital deployed on it, exceeds the data's licensing cost. Suppose a fund would deploy a \$50 million position whenever Section 17.2.4's signal fires long, which happened in five of the eight quarters of Table 17.3 below, and that the signal's incremental edge over an unconditional baseline is the 0.5425 percentage points computed there (the 0.88% average return on signaled quarters minus the 0.3375% average across all quarters). With four earnings releases per year and a 5/8 signal frequency, the fund expects about $4 \times 5/8 = 2.5$ signaled events a year, for an expected incremental profit of $\$50\text{M} \times 0.54\% \times 2.5 \approx \$678{,}000$. Against Section 17.6's subscription-cost range, a satellite feed at \$250,000 per year plus a web and card-panel bundle at \$150,000, data costs total \$400,000, for a net expected benefit of about \$278,000 a year. The edge is modest, and Section 17.3.3 warns that it will shrink as more funds license the same feeds. Even the gross figure flatters the signal: the 0.5425-point edge is measured on the same eight quarters the regression was fitted to, so \$678,000 is an in-sample upper bound on what the signal would earn on quarters it has not seen.

The same arithmetic drives a build-versus-buy decision many larger funds eventually face. Once trading volume across many similar signals is large enough, an in-house satellite-analytics or web-scraping team can be cheaper than paying a vendor's margin on the same imagery or logs, at the cost of taking on the computer-vision and data-engineering work Section 17.4 describes. Smaller funds typically lack the scale to justify this fixed cost and remain vendor customers, one more way in which alternative data's advantages concentrate among larger participants.

17.2.3 Worked Example 17.1: Satellite Imagery and Retailer Revenue

Suppose a hypothetical home-improvement retailer, Cascade Home & Garden, is followed by a data vendor that counts cars in a sample of its store parking lots from satellite imagery, as Section 17.6 describes being done for real retailers since 2010. Table 17.2 lists eight quarters of year-over-year car-count growth alongside Cascade's subsequently reported year-over-year revenue growth.

Fitting the simple regression Revenue growth $= \alpha + \beta\,$(Car-count growth) by ordinary least squares, following Chapter 6's Section 6.3.1 method, gives $\bar{x} = 1.7125$, $\bar{y} = 1.6625$, $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y}) = 43.92375$, $S_{xx} = \sum(x_i - \bar{x})^2 = 67.28875$, so $\hat{\beta} = S_{xy}/S_{xx} = 0.6528$ and $\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x} = 0.5446$.

The fitted line, $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528\,(\text{Car-count growth})$, achieves $R^2 = 0.870$: car-count growth alone explains 87% of the quarter-to-quarter variation in Cascade's revenue growth in this sample. That is strong for a single proxy, though 13% of the variation remains unexplained.

Suppose satellite data for a ninth quarter shows 5.0% car-count growth three weeks before Cascade's earnings release. The regression predicts $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528(5.0) = 3.809\%$, a tradeable estimate of a number the market will not see for three more weeks.

Wulin.Suo@Queensu.ca

Quarter	Car-count growth (%)	Revenue growth (%)
Q1	2.1	1.5
Q2	−1.5	−0.3
Q3	4.8	4.5
Q4	0.6	1.0
Q5	6.2	4.0
Q6	−3.0	−1.0
Q7	3.5	3.8
Q8	1.0	−0.2

Table 17.2: Satellite car-count growth and Cascade's reported revenue growth, by quarter

17.2.4 Combining Alternative Data Sources: A Multi-Source Earnings Signal

A real alternative-data desk rarely uses a single source. It combines several into one model, to average out each source's idiosyncratic noise and to capture aspects of the business a single proxy misses. Figure 17.1 sketches the resulting pipeline.

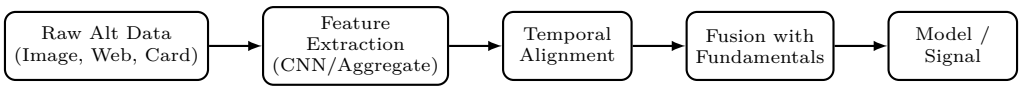


Figure 17.1: The alternative-data pipeline this chapter develops

Extending Section 17.2.3's example, suppose Cascade is also tracked by a web-traffic vendor (year-over-year growth in visits to Cascade's e-commerce site) and a card-panel vendor (year-over-year growth in aggregated card spending at Cascade's stores). Table 17.3 lists all three alternative-data growth rates alongside the earnings surprise (actual revenue growth minus the analyst consensus available at the time) and Cascade's stock return on the trading day after the announcement.

Quarter	Car (%)	Web (%)	Card (%)	Surprise (%)	Fitted (%)	Signal	Return (%)
Q1	2.1	3.0	1.5	0.5	0.48	Long	1.1
Q2	−1.5	−0.5	−1.0	−1.3	−1.08	Flat	−0.6
Q3	4.8	5.5	3.0	2.5	1.95	Long	2.0
Q4	0.6	1.5	0.8	0.0	−0.44	Flat	0.3
Q5	6.2	4.0	4.5	2.0	2.46	Long	−0.4
Q6	−3.0	−2.0	−1.8	−1.5	−1.92	Flat	−1.4
Q7	3.5	2.5	2.0	2.3	1.66	Long	1.5
Q8	1.0	2.0	0.5	−1.2	0.18	Long	0.2

Table 17.3: Multi-source alternative data, earnings surprise, and next-day return, by quarter

Regressing surprise on the three alternative-data growth rates by ordinary least squares, using the normal equations

$$\hat{\beta} = (X^\top X)^{-1} X^\top y, \tag{17.1}$$

with X's columns an intercept, car-count growth, web-traffic growth, and card growth, gives

$$\hat{\beta} = (-0.291,\ 1.092,\ -0.113,\ -0.792), \qquad R^2 = 0.837.$$

The negative coefficient on card growth is not a mistake. With car-count and web-traffic growth already in the regression, card growth's explanatory power is largely redundant, and its small residual coefficient partly offsets collinear noise the other two share. This is common when several correlated proxies for the

Wulin.Suo@Queensu.ca

same quantity enter one regression, and a reminder that individual coefficients in a multi-source model are not clean, isolated effects the way Chapter 6's Section 6.3.1 single-variable coefficient is.

Column "Fitted" reports each quarter's fitted surprise, and column "Signal" converts a positive fitted surprise into a long position taken ahead of the release (flat otherwise), the logic Chapter 16's Section 16.3.1 applied to a sentiment score. Five of the eight quarters signal long; four of those (Q1, Q3, Q7, Q8) are followed by a positive next-day return and one (Q5) by a negative return despite a strongly positive fitted surprise, an 80% hit rate.

The signaled quarters average a 0.88% next-day return versus 0.34% across all eight quarters. The edge is real within this sample, though in-sample, since it is measured on the quarters the regression was fitted to, and it is probabilistic, as Chapter 6 and Chapter 16's Section 16.3.1 both emphasize: Q5's miss shows that even a well-fitted multi-source model cannot rule out a price reaction driven by something the data could not capture, such as guidance issued alongside the release. A ninth quarter with car, web, and card growth of 5.0%, 4.5%, and 3.5% yields a fitted surprise of

$$-0.291 + 1.092(5.0) - 0.113(4.5) - 0.792(3.5) = 1.888\%,$$

again a tradeable estimate available before the official release.

17.3 Working with Alternative Data

Alternative data breaks alignment, scale, and quality assumptions that price data never did.

17.3.1 Point-in-Time Alignment and the Cost of Look-Ahead Bias

Section 17.2.4's regression silently assumed that every growth rate in Table 17.3 was in hand before that quarter's earnings release. Card-panel data shows why that assumption fails even when delivery is fast. Commercial card panels typically arrive within days or weeks of the period they cover, but the history a vendor sells today is not the history a subscriber saw in real time. Late transactions are added after the first release, the panel's composition shifts as card issuers join or leave, and vendors routinely revise and re-weight past figures, backfilling the entire history with today's panel and methodology. A backtest that treats today's revised series as if each figure had been known in that form at the time commits look-ahead bias: it uses information it could not have had.

To see how much this matters, refit the regression on quarters Q2 through Q8 (dropping Q1, which has no prior quarter to lag from) two ways. The naive version uses each quarter's contemporaneous card growth, as before:

$$\hat{\beta}_{\text{naive}} = (-0.290,\ 1.102,\ -0.119,\ -0.802), \qquad R^2 = 0.837,$$

with four quarters signaling long (Q3, Q5, Q7, Q8) and a 75% hit rate. The ideal fix is each quarter's first-release vintage, the figure as originally delivered, but a backtest often lacks those vintages. A common conservative substitute, used here, is the card growth reported one quarter earlier, a figure certainly in hand before the release:

$$\hat{\beta}_{\text{realistic}} = (0.251,\ 0.424,\ -0.100,\ -0.286), \qquad R^2 = 0.873,$$

but Q8 no longer signals long once the lag is respected, leaving three quarters (Q3, Q5, Q7) with a 66.7% hit rate and a higher average return (1.03% versus 0.83%) on that smaller, more selective set of trades.

The lesson is not that the lagged version is worse (its R^2 and its average signaled return are both higher) but that it is a *different* model making *different* trading decisions on the same quarters. A backtest of the naive version reports performance for trades the fund could never have placed, since the card figures they depend on were not yet in hand in that form, whether that backtest happens to look better or worse. This is why Section 17.3.3 treats delivery timing and revisions as data-quality issues rather than modeling nuances. The direction of the bias is not predictable in general, but ignoring delivery timing and data vintages always invalidates a backtest as a measure of tradeable performance. Point-in-time discipline, using for every historical date only the data vintage available on that date, is the standard defense.

Wulin.Suo@Queensu.ca

17.3.2 Feature Engineering for Alternative Data

Raw alternative-data readings, like the market and fundamental data of Chapters 2 through 5, are rarely used as delivered. Section 17.3.1 addressed when a reading becomes usable; this section addresses what must be done to it first.

Seasonal adjustment. Web traffic, foot traffic, and card volumes carry strong calendar effects (day-of-week patterns, holiday spikes, back-to-school and year-end seasons) unrelated to a firm's underlying performance. Comparing a reading with the same period a year earlier, as year-over-year growth rates such as Section 17.2.3's already do, removes most of this, provided the calendar aligns. Thanksgiving week, for instance, falls on a different date each year, so the alignment should be by trading day or holiday-relative position rather than by calendar date.

Peer-relative normalization. A retailer's car-count or web-traffic growth reflects both firm-specific performance and sector-wide conditions (gasoline prices, consumer spending, weather). Subtracting a sector-average benchmark isolates the firm-specific component a stock-picking strategy cares about.

Suppose sector-average car-count growth for Cascade's peers over the eight quarters of Table 17.2 is $(1.0, -0.5, 2.0, 0.3, 2.5, -1.0, 1.5, 0.5)$; subtracting it from Cascade's own gives a peer-adjusted series $(1.1, -1.0, 2.8, 0.3, 3.7, -2.0, 2.0, 0.5)$. Refitting the regression on this series gives $\hat{\beta} = 1.064$, $\hat{\alpha} = 0.678$, and $R^2 = 0.867$, essentially unchanged from the raw series' $R^2 = 0.870$. Sector-wide movements were small relative to Cascade's own variation in this sample, so the adjustment barely changes the fit. It matters far more when the whole sector moves together, in a consumer-spending downturn or a fuel-price spike that depresses driving-based foot traffic, where an unadjusted signal would mistake a sector-wide move for firm-specific news.

17.3.3 Data Quality, Bias, and Integration Challenges

Alternative data's advantages (frequency, granularity, and an informational edge before official reporting) come with quality problems that have no close analogue in the market and fundamental data earlier chapters used.

Coverage bias. A satellite vendor can cover only stores it has usable imagery for, typically larger, freestanding locations with visible parking lots, so its sample underrepresents smaller-format or mall-based stores, and a model trained on the covered subset may not generalize to the retailer as a whole.

Vendor black-box processing. As Section 17.2.1 noted, a vendor typically sells an already-processed feature, so an analyst inherits whatever computer-vision or aggregation errors are baked into its pipeline without being able to inspect them. An undisclosed methodology change between periods can look, in the data, exactly like a change in the underlying business.

Delivery lag and look-ahead bias. A backtest that assumes alternative data was available on the date it describes, rather than the often later date it reached subscribers, uses information that was not tradeable in real time. Section 17.2.4's multi-source signal is useful only if all three sources reach the desk before the earnings release, not merely before the quarter ends, and only in the vintage that arrived then, not as later revised.

Licensing cost and alpha decay. High-quality alternative data is expensive and often licensed exclusively or semi-exclusively. Section 17.6 documents a 4-5% informational edge around earnings from satellite car counts; as more funds license the same feed, that edge is competed away, the erosion of a widely traded pattern Chapter 6 described. An alternative-data edge has a shelf life.

Privacy and legal exposure. Card-panel and geolocation data are aggregated and anonymized before sale, but reconstructing individual behavior from supposedly anonymized data has repeatedly proven possible in other domains, so a vendor's privacy practices, and a buyer's own legal exposure, are due-diligence questions that market and fundamental data never raised.

The line between alternative data and material non-public information is not always bright. Card-panel and web-traffic vendors aggregate across many cardholders or visitors so that no single company's non-public information is sold, but the boundary is a legal judgment, not a technical threshold. A narrowly scoped feed, transaction data from only a handful of a firm's largest customers, say, can approach material non-public information even though it never touches the firm's systems and no insider leaks anything. This is a new compliance risk that Chapter 15's insider-trading discussion, framed around information leaking from inside a firm, did not anticipate.

Wulin.Suo@Queensu.ca

Is this signal causal? Car counts do not cause revenue, and Section 17.2.3's regression never claimed they did: both are effects of customers visiting stores. A trading signal needs only prediction, the job Chapter 6's Section 6.4.6 assigns to predictive models. The causal question still matters, for two reasons. The first is durability. A proxy that predicts because it measures the business (cars in the lot are shoppers) should keep working as long as the business model does. One that predicts because a third variable drove both it and the target during the sample, a run of fine spring weather lifting both foot traffic and garden-center sales, say, is exposed to the confounding Section 6.4.6 describes, and fails the first time that variable moves on its own. Asking what mechanism links a signal to cash flows is the cheapest robustness test available, and a signal with no plausible mechanism, found by screening many series, is more likely the multiple-testing artifact of Section 17.2.2. The second reason is intervention. A retailer that reads the same car counts to decide where to open stores or how much to spend on promotions is asking a causal question the regression cannot answer, for the same reason that Chapter 6's credit-limit flag predicted lower default while causing higher default. As with Chapter 7's Granger causality, a signal that predicts carries information; it says nothing about how the information arises.

17.4 Neural Networks for Structured and Unstructured Data

Section 17.2.3 treated the vendor's car count as a finished number, but the vendor had to produce it from raw satellite pixels, and the web-traffic and card figures likewise start from raw logs or transaction records. Chapter 6's feedforward network, in which every input feeds every hidden unit with its own weight, ignores two kinds of structure alternative data routinely has: the two-dimensional grid of an image and the ordering of a time-stamped log. Specialized architectures exploit each, and a third needs no labeled target at all.

17.4.1 Convolutional Neural Networks for Grid-Structured Data

Suppose a vendor wants a model to count cars in a million-pixel satellite tile rather than paying an analyst to do it by eye. A feedforward network could treat each pixel as an input feature, but it would need a separate weight connecting each pixel to each hidden unit, and it would have to learn separately that a cluster of bright pixels in the top-left corner and the identical cluster in the bottom-right both mean a car, since nothing in its structure says that a pattern's meaning does not depend on where it sits.

A convolutional neural network (CNN) exploits this redundancy. A useful pattern (a car's outline, an edge, a texture) can appear anywhere in the frame, so instead of a separate weight for every pixel-hidden-unit pair, a CNN learns a small filter (or kernel) and slides it across the image, reusing the same weights at every position. This weight sharing makes CNNs practical: a 1000×1000-pixel tile has a million pixels, far too many for per-pixel weights to be learned from any realistic training set, but a 2×2 filter has four weights regardless of image size.

Consider a stylized 4×4 satellite image patch of a single parking lot, with pixel intensities between 0 (dark asphalt) and 1 (a car's bright roof), containing two clusters of parked cars in opposite corners:

$$I = \begin{pmatrix} 0.1 & 0.1 & 0.8 & 0.9 \\ 0.1 & 0.1 & 0.9 & 0.8 \\ 0.7 & 0.8 & 0.1 & 0.1 \\ 0.8 & 0.9 & 0.1 & 0.1 \end{pmatrix}.$$

Convolving this image with a 2×2 averaging filter

$$K = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}$$

at stride 1 (sliding the filter one pixel at a time, computing the elementwise product with each 2×2 patch and summing) produces a 3×3 feature map. The top-left entry, for instance, averages the patch

$$\begin{pmatrix} 0.1 & 0.1 \\ 0.1 & 0.1 \end{pmatrix}$$

Wulin.Suo@Queensu.ca

to 0.1; the entry one column to the right averages

$$\begin{pmatrix} 0.1 & 0.8 \\ 0.1 & 0.9 \end{pmatrix}$$

to $(0.1 + 0.8 + 0.1 + 0.9)/4 = 0.475$. Carrying this out for all nine positions gives

$$\text{Feature map} = \begin{pmatrix} 0.100 & 0.475 & 0.850 \\ 0.425 & 0.475 & 0.475 \\ 0.800 & 0.475 & 0.100 \end{pmatrix}.$$

The two corner entries where a car cluster sits, 0.850 (top right) and 0.800 (bottom left), stand out above the two empty corners (0.100 each) and the mixed middle entries: the filter highlights regions of concentrated brightness.

A trained CNN learns many such filters at once (edge, corner, and texture detectors) and stacks several convolutional layers, so that later layers respond to increasingly complex combinations of earlier features. It typically applies a ReLU activation and a pooling step (taking, say, the maximum of each 2×2 block) after each convolution, shrinking the map while keeping its strongest responses, before feeding the result into feedforward layers of the kind Chapter 6 described to produce a single number, an estimated car count.

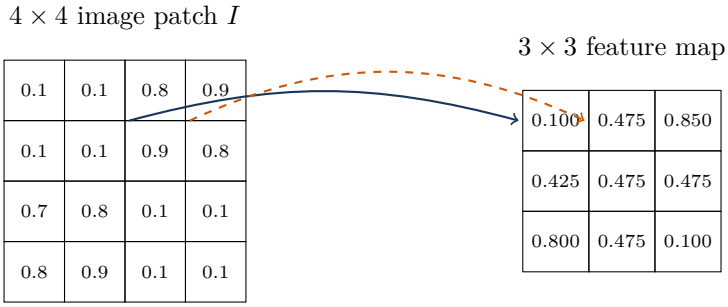


Figure 17.2: Sliding a 2×2 averaging filter across the 4×4 satellite patch I at stride 1

Stacking convolutional layers naively runs into a training problem once a network gets deep: gradients computed by backpropagation (Chapter 6) must flow back through every layer, and in a deep plain CNN they can shrink toward zero before reaching the earliest layers, leaving those layers essentially untrained. This is the vanishing-gradient problem.

The residual network (ResNet), the architecture that made very deep CNNs practical, adds a shortcut connection carrying each block's input to its output. The block then learns only a residual adjustment to what passes through unchanged, and gradients flow through the shortcut even when the block's own transformation contributes little. This is the instinct behind Chapter 12's shrinkage methods (do not force a model to learn from nothing what it could inherit and adjust), applied to network depth rather than portfolio weights.

17.4.2 Recurrent Neural Networks for Sequential Data

Suppose an analyst wants to judge whether Cascade's web traffic is accelerating into its next earnings release from the last several weeks of readings, since a single week cannot distinguish a steady climb from a spike about to reverse. A feedforward network could take the weeks as separate features, but it would treat them as an unordered bag, with nothing telling it that three weeks ago came before last week.

A recurrent neural network (RNN) addresses this. It processes a sequence one step at a time, maintaining a hidden state h_t that is passed forward and updated at every step,

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b), \tag{17.2}$$

so that h_t is, in principle, a function of every input seen so far, $x_1, \ldots, x_t$. This gives the network a memory that Chapter 7's autoregressive and GARCH recursions share in spirit (each defines a current value as a function of a prior state plus new information) and that Chapter 6's feedforward network lacks.

Wulin.Suo@Queensu.ca

Suppose Cascade's e-commerce site shows three successive weekly readings of normalized web-traffic growth in the final three weeks before its next earnings release, $x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, and a single-hidden-unit RNN with weights $w_x = 0.6$, $w_h = 0.4$, $b = 0.1$, and initial hidden state $h_0 = 0$. Step by step:

$$h_1 = \tanh(0.6 \times 0.5 + 0.4 \times 0 + 0.1) = \tanh(0.400) = 0.3799,$$
$$h_2 = \tanh(0.6 \times (-0.2) + 0.4 \times 0.3799 + 0.1) = \tanh(0.13196) = 0.1312,$$
$$h_3 = \tanh(0.6 \times 0.9 + 0.4 \times 0.1312 + 0.1) = \tanh(0.69248) = 0.5996.$$

Notice that h_2 is close to zero not because x_2 was extreme (it was mildly negative) but because it also carries a damped memory of h_1: the hidden state blends past and present rather than reacting to the latest input alone.

Passing the final hidden state through an output layer, $\hat{y} = \sigma(w_y h_3 + b_y)$, where $\sigma(z) = 1/(1 + e^{-z})$ is Chapter 6's sigmoid, with $w_y = 1.2$ and $b_y = -0.3$, gives $\hat{y} = \sigma(1.2 \times 0.5996 - 0.3) = \sigma(0.4195) = 0.6034$, which might be read as a 60.3% estimated probability that Cascade's upcoming quarter shows accelerating revenue growth. Figure 17.3 unrolls this computation across its three time steps, showing how each hidden state depends on the current input and the previous hidden state.

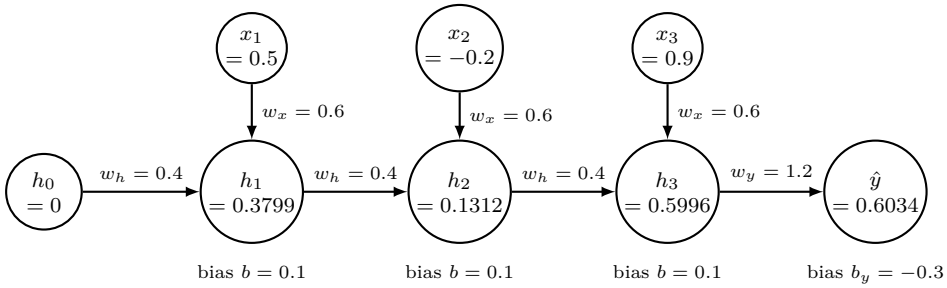


Figure 17.3: The single-hidden-unit RNN unrolled across its three time steps

In practice, a plain RNN's memory decays quickly over long sequences, since each step multiplies an early input's running influence by another factor less than one. The long short-term memory (LSTM) architecture addresses this by maintaining a separate cell state c_t alongside the hidden state, updated as

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{17.3}$$

where

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

is a forget gate controlling how much of the previous cell state to retain,

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

is an input gate controlling how much of a new candidate value

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

to admit, and $\odot$ denotes elementwise multiplication; a third, output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

then controls how much of c_t is exposed as the hidden state

$$h_t = o_t \odot \tanh(c_t). \tag{17.4}$$

The three gates use σ, whose output lies in $(0, 1)$ and reads as a fraction retained or admitted; the candidate uses tanh, whose output lies in $(-1, 1)$ and carries a signed value to write into the cell state. The worked

Wulin.Suo@Queensu.ca

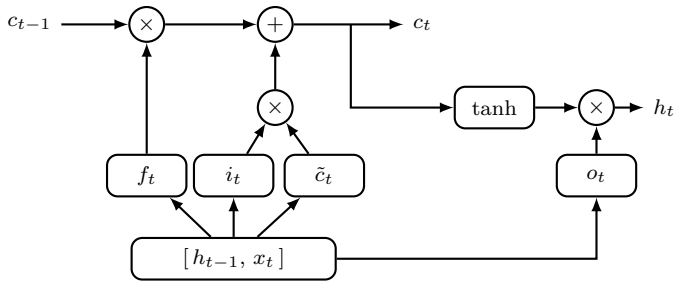


Figure 17.4: The LSTM cell. The cell state crosses the top of the diagram scaled once and added to once, never passed through a squashing function, which is why information survives along it; the three gates below all read the same concatenated $[h_{t-1}, x_t]$.

example below turns on that difference: from identical arguments it computes $i_1 = \sigma(0.25) = 0.5622$ but $\tilde{c}_1 = \tanh(0.25) = 0.2449$. Figure 17.4 puts these six equations in one picture, with the cell state running along the top and the gates feeding it from below.

Because the cell state's update is additive rather than fully multiplicative, a forget gate near 1 lets information pass across many time steps largely undamped, fixing the memory decay the single-unit example illustrates. The cost is four times as many weights as the plain recurrent unit, one set each for the forget, input, and output gates and the candidate.

A worked example makes this concrete. Apply a single-unit LSTM to the plain RNN's three-week input sequence ($x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, $h_0 = 0$, $c_0 = 0$), with illustrative gate weights $w_{f,h} = 0.5$, $w_{f,x} = 0.3$, $b_f = 2.0$ (a large bias, chosen so the forget gate stays close to 1 and the memory effect is visible) and $w_{i,h} = w_{i,x} = w_{c,h} = w_{c,x} = w_{o,h} = w_{o,x} = 0.5$ with all other biases at 0. The first step gives

$$f_1 = \sigma(2.15) = 0.8957, \qquad i_1 = \sigma(0.25) = 0.5622, \qquad \tilde{c}_1 = \tanh(0.25) = 0.2449,$$
$$c_1 = f_1 c_0 + i_1 \tilde{c}_1 = 0.1377, \qquad o_1 = \sigma(0.25) = 0.5622, \qquad h_1 = o_1 \tanh(c_1) = 0.0769.$$

Continuing this recursion through the second and third steps gives

$$f_2 = 0.8785, \qquad c_2 = 0.0912, \qquad h_2 = 0.0441,$$
$$f_3 = 0.9082, \qquad c_3 = 0.3537, \qquad h_3 = 0.2092;$$

passing h_3 through the same output layer as the plain RNN example ($w_y = 1.2$, $b_y = -0.3$) gives $\hat{y} = \sigma(1.2 \times 0.2092 - 0.3) = 0.4878$.

The forget gates $f_2 = 0.8785$ and $f_3 = 0.9082$ are the mechanism to isolate. Their product, $0.8785 \times 0.9082 \approx 0.7979$, is the fraction of c_1 that survives two steps later through the forget gate's multiplicative decay, against $w_h^2 = 0.4^2 = 0.16$ for the plain RNN's two-step decay of its hidden state's dependence on h_1.

Roughly 80% of c_1's information survives to c_3 in the LSTM, against roughly 16% in the plain RNN. Information still decays in an LSTM, since the forget gate is strictly less than 1, but a trained forget gate can sit close enough to 1 that memory persists over many more steps than a plain RNN's fixed recurrent weight allows. Figure 17.5 draws the first step as a cell diagram, with all four gates computed from the same inputs before they combine into the new cell and hidden states.

17.4.3 Autoencoders for Unsupervised Anomaly Detection

Suppose a data-quality analyst wants to flag quarters whose alternative-data readings look unusual, without defining "unusual" in advance and without a single labeled example of a bad reading. The CNN and RNN examples cannot address this unsupervised problem, since both are trained toward a known target.

An autoencoder is trained without any label. It learns an encoder that compresses an input to a small bottleneck representation and a decoder that reconstructs the input from it, with reconstruction error as the only training signal. Trained on typical data, it reconstructs anomalous inputs poorly, since its bottleneck

Wulin.Suo@Queensu.ca

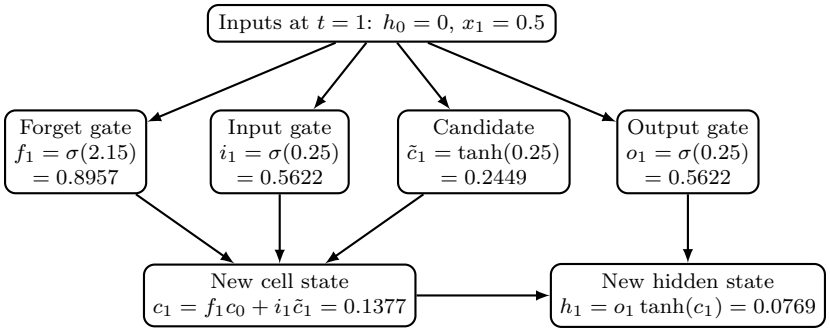


Figure 17.5: The LSTM cell's first time step, computed from the worked example in the text

was never shaped to represent what made them unusual. Reconstruction error is thus a natural anomaly score, extending Chapter 15's classification-based anomaly detection to a setting with no labeled anomalies.

Consider a simple linear autoencoder, encoder weights $w = (0.5, 0.3, 0.2)$ applied to Table 17.3's three alternative-data growth rates $x = (\text{car}, \text{web}, \text{card})$ for each quarter, producing a single bottleneck code $z = w \cdot x$, and a tied-weight decoder that reconstructs $\hat{x} = z\,w$. For Q4, $x = (0.6, 1.5, 0.8)$, so $z = 0.5(0.6) + 0.3(1.5) + 0.2(0.8) = 0.91$, and $\hat{x} = 0.91(0.5, 0.3, 0.2) = (0.455, 0.273, 0.182)$, giving a reconstruction error $\sum(x_i - \hat{x}_i)^2 = (0.6 - 0.455)^2 + (1.5 - 0.273)^2 + (0.8 - 0.182)^2 = 1.908$.

Carrying this out for all eight quarters and dividing each quarter's error by its total energy $\sum x_i^2$ (so that quarters with larger growth rates are not flagged just for being larger) gives relative errors from 39.4% (Q7) to 58.7% (Q4). Q4 is the quarter the autoencoder reconstructs worst, not because its numbers are largest (Q5's raw error of 30.9 is far larger) but because Q4's web-traffic growth (1.5%) is large relative to its car-count and card growth (0.6% and 0.8%), a pattern this bottleneck direction does not represent well.

One caution: the weights $w = (0.5, 0.3, 0.2)$ are illustrative, chosen for easy arithmetic, not trained. A trained tied-weight linear autoencoder learns a unit-length weight vector (the data's first principal direction), and only a unit-length w makes $\hat{x} = z\,w$ a true projection. This w has length 0.6164, so every reconstruction above is shrunk toward zero and every error inflated. Rescaling w to unit length, $(0.8111, 0.4867, 0.3244)$, cuts the relative errors to between 1.6% (Q7) and 32.9% (Q4); the levels fall sharply, but Q4 is still the worst-reconstructed quarter, which is the point of the example.

This is the quarter a data-quality analyst should investigate first (is the web-traffic reading an error, a one-time promotional spike, or an early signal the other two sources have not yet picked up?) before trusting it in a downstream model.

Every network in this section runs on hand-picked weights and a single forward pass, so that each number can be checked by hand. The companion notebook's optional "at scale" section trains the real thing in PyTorch: an LSTM forecaster validated walk-forward against simpler baselines and an autoencoder that rediscovers the unit-length direction above from 3,000 simulated rows; it needs the extra packages listed in the repository's `requirements-dl.txt`.

17.4.4 Transformers as a Unifying Architecture

Chapter 16 developed the transformer and its self-attention mechanism (Section 16.4.1) for text, where each token attends to every other token. The mechanism generalizes: a vision transformer (ViT) divides an image into fixed-size patches, treats each patch as a token, and applies self-attention across patches instead of convolution across pixels. A satellite image classifier can then weigh a distant part of the image against the current patch directly, which a CNN's local filters do only through many stacked layers.

The companion notebook splits the CNN example's 4×4 image into four non-overlapping 2×2 patches, treats each as a token, and computes scaled dot-product self-attention across them. Querying from patch 2 (the top-right car cluster, mean brightness 0.85) against keys built from each patch's brightness, patch 2 attends to itself with weight 0.6174 and to patch 3 (the bottom-left car cluster, mean brightness 0.80, the

Wulin.Suo@Queensu.ca

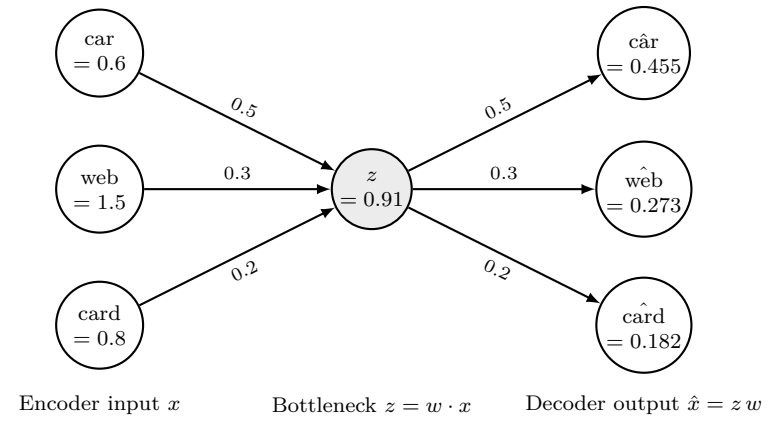


Figure 17.6: The linear autoencoder applied to Q4's three alternative-data growth rates

farthest patch in the image) with weight 0.3817, while the two empty patches 1 and 4 (mean brightness 0.10 each) receive a combined weight of only 0.001.

Patch 2's most relevant neighbor is thus patch 3, a content-similar patch in the opposite corner, not either empty patch nearer to it, and self-attention finds that relevance in one step rather than through several stacked convolutional layers. In current practice CNNs remain common for smaller or highly structured image tasks (including much car-counting and object-detection work), while transformer-based architectures increasingly dominate large-scale image classification and the web-traffic and card-transaction sequence modeling that once used RNNs. A single architectural family, self-attention, now spans text (Chapter 16), images, and sequences.

Training discipline for deep learning in finance. Every network in this section has a handful of weights; a production LSTM or CNN has tens of thousands to millions, fitted to short, noisy, nonstationary financial data. Four habits keep such a model honest. *Dropout* sets a random fraction of hidden units (commonly 10–50%) to zero at each training step, so that no unit can lean on a particular partner and the trained network behaves like an average of many thinner ones; it is switched off at prediction time. *Weight decay* adds Chapter 6's ridge penalty to the loss, shrinking every weight toward zero. *Early stopping* records the loss on a held-out validation block after each pass through the training data and keeps the weights from the epoch where it was lowest, since the training loss keeps falling long after the network has started to fit noise. *Walk-forward splits* make every validation and test block strictly later than the training data, as Chapter 7 requires of any forecast. Random folds let the network learn from the future of the periods it is scored on, and labels that overlap in time call for a gap between training and test blocks as well (López de Prado 2018).

The companion notebook's optional LSTM cell applies the last two habits to 1,500 simulated observations whose dynamics switch between momentum after a rise and reversal after a fall, a nonlinearity no linear autoregression can represent. Averaged over three walk-forward test blocks, the committed run's root-mean-squared errors are 1.128 for a naive last-value forecast, 1.074 for a linear AR(2), 1.066 for gradient-boosted trees, and 1.050 for the LSTM, against a noise floor of 1.037 that even the true model cannot beat. The LSTM wins by 2.3% over the AR(2) on a problem built so that a linear model must fail. A rerun on another machine gives similar rather than identical digits, since a trained network's weights depend on floating-point details that seeding does not fix.

Even that narrow win is the exception for the data shape most finance problems arrive in, a table of features per firm, borrower, or transaction. Comparing deep networks with tree ensembles on 45 tabular datasets of around 10,000 rows each, after extensive tuning of both, Grinsztajn et al. (2022) found that random forests and gradient-boosted trees remained stronger. They traced the gap to three properties of tabular data: many uninformative features, which trees ignore and networks must learn to ignore; columns whose individual meaning matters, which a network's indifference to rotations of the feature space throws away; and targets with sharp, irregular jumps, which a tree fits with one split and a network smooths over.

Wulin.Suo@Queensu.ca

Chapter 21's real-data credit bake-off, where the two tree ensembles lead, is the typical finance case. The exception is return prediction at very large scale: in the Gu, Kelly, and Xiu comparison Chapter 12 discusses, on tens of thousands of stocks over six decades, shallow neural networks edged out the tree ensembles (Gu et al. 2020). The working rule: start with gradient boosting on tabular data, and reach for a deep network when the input is an image, a sequence, or text, or when the dataset is large enough to pay for one.

17.5 Applications and Advanced Methods

This part builds a signal from disclosure text, then surveys the deep-learning methods the data invites.

17.5.1 Building an ESG Signal Directly from Disclosure Text

Chapter 12 showed that six major ESG rating providers agree only weakly (average pairwise correlations of 38% to 71%), mostly because they measure different facts about a company and weight them differently, not because they measure an agreed concept noisily. Section 17.2.2's build-versus-buy logic applies here as it does to satellite imagery: rather than buying a vendor's ESG score and inheriting its undisclosed methodology, a fund can build a narrow, transparent environmental-tone signal from a company's public disclosures with the text-analysis tools of Chapter 16.

Suppose Meridian Fabrication, the manufacturer whose ratios Chapter 4 analyzed, files a 10-K whose risk-factors section runs to several pages. Chapter 16's Naive Bayes classifier (Section 16.3.2) can be extended with a third class, Environmental Risk, trained on a few sentences using vocabulary (emissions, environmental, climate, remediation) that the original Credit Risk and Market Risk training sentences never mention. The retrained classifier flags four sentences in Meridian's filing as Environmental Risk with high confidence, each keying on at least one environment-specific term:

> S1: "Management remains confident that planned facility upgrades will reduce our emissions, but continued regulatory headwinds could still increase compliance costs."
> S2: "We remain cautious about the pace of emissions reduction given persistent supply chain headwinds affecting our environmental capital projects."
> S3: "Emissions declined for the third consecutive year, reflecting strong progress on our environmental targets despite a challenging regulatory backdrop."
> S4: "Physical risks from climate change, including potential flooding at our coastal facility, remain a weak point in our disaster preparedness planning and a continuing concern for the board."

Applying Chapter 16's Loughran-McDonald dictionary tone score (Section 16.2.3) to the four sentences, with the same positive word set {strong, growth, exceeded, confident, improved} and negative word set {cautious, headwinds, decline, declined, weak, concern, concerns} Chapter 16 used to score its two earnings-call transcripts, gives

$$\begin{aligned}
&\text{S1: } (1-1)/20 = 0.0,\\
&\text{S2: } (0-2)/19 \approx -0.1053,\\
&\text{S3: } (1-1)/19 = 0.0,\\
&\text{S4: } (0-2)/28 \approx -0.0714,
\end{aligned}$$

an average environmental tone score of -0.0442 across the four sentences, a mildly cautious reading of Meridian's own environmental disclosure.

This exercise surfaces a new data-quality problem specific to building ESG signals from generic financial-sentiment tools. The Loughran-McDonald lexicon was built to score revenue and earnings language, where "declined" is reliably bad news, but in sentence S3 the same word describes good environmental news, emissions falling for a third consecutive year, and the lexicon cannot tell the difference. S3's score of 0.0 understates how positive the sentence is. Fixing this needs a lexicon built for environmental disclosure, one that distinguishes a decline in emissions from a decline in revenue, and no standardized public dictionary of that kind yet exists the way Loughran-McDonald's does for general financial tone.

Wulin.Suo@Queensu.ca

Comparing this result with Chapter 12's worked example makes the deeper point. That example scored three fictional assets on a common, if arbitrary, 0-to-100 composite scale and varied only the *weight* providers place on the same E, S, and G sub-scores, one of three sources of divergence Berg, Kölbel, and Rigobon identify. This section's -0.0442 tone score is not on that scale and not comparable to it: it measures something narrower (the tone of one filing's risk-factors section) by a different method (word counting rather than a proprietary composite), the *scope* and *measurement* divergence the same paper finds behind most of the disagreement. An in-house signal like this one does not resolve the disagreement; it adds a fourth, differently scoped measurement to a field that already has too many inconsistent ones.

What it buys instead is auditability: every step from raw sentence to final number can be inspected, recomputed, and defended to a model validator, as Chapter 8's model-risk-management framework requires.

17.5.2 Advanced Topics: Transfer Learning, Graphs, and Fusion

Four further techniques extend Section 17.4's toolkit in directions that matter increasingly in practice: reusing an already-trained model instead of training from scratch, exploiting network structure between entities, combining several data modalities into one representation, and pretraining without labels.

Transfer Learning: Fine-Tuning a Pretrained Vision Backbone

Section 17.3.3 noted that a new alternative-data feed typically has a short history, too little to train a large CNN's millions of weights from scratch. Transfer learning sidesteps this: a CNN already trained on a large general image dataset, which has learned generic filters for edges, textures, and shapes, is reused as a frozen feature extractor, and only a small final layer mapping its features to the task (car count, cloud cover, construction status) is trained on the limited task data. Because the frozen backbone receives no updates, this training is Chapter 6's gradient descent applied to far fewer parameters than a full network has.

Suppose a pretrained backbone, frozen, produces a two-dimensional feature vector $f = (0.7, 0.3)$ for a new satellite image patch, and the task is to fine-tune a final logistic layer, currently $w = (0.4, 0.4)$, $b = -0.1$, to predict whether the patch contains a car cluster (label $y = 1$ for this patch). The current prediction is

$$\hat{y} = \sigma(w \cdot f + b) = \sigma(0.4 \times 0.7 + 0.4 \times 0.3 - 0.1) = \sigma(0.30) = 0.5744.$$

Following Chapter 6's gradient descent update with learning rate $\eta = 0.5$, the gradient of the log-loss with respect to w is

$$(\hat{y} - y)f = (0.5744 - 1)(0.7, 0.3) = (-0.298, -0.128),$$

and with respect to b is $\hat{y} - y = -0.426$, giving updated weights

$$w \leftarrow (0.4, 0.4) - 0.5(-0.298, -0.128) = (0.549, 0.464)$$

and

$$b \leftarrow -0.1 - 0.5(-0.426) = 0.113.$$

Recomputing the prediction with these updated weights gives

$$\hat{y} = \sigma(0.549 \times 0.7 + 0.464 \times 0.3 + 0.113) = \sigma(0.6365) = 0.6540,$$

closer to the true label of 1 after a single gradient step on a handful of parameters. This is why a fund with only a few hundred labeled satellite patches can still fine-tune a usable classifier, rather than needing the millions of labeled images a CNN trained from scratch would require.

Graph Neural Networks for Ownership and Counterparty Networks

Chapter 15 modeled a shell-company layering network as a static sequence of hops. A graph neural network (GNN) treats the network as a computational structure: each node's representation is updated by aggregating its neighbors', so risk propagates through the network instead of being assessed for each entity in isolation. This is the structure alternative data increasingly captures (common addresses, shared directors, correlated

Wulin.Suo@Queensu.ca

transaction timing) and a purely tabular model discards. A single GNN layer updates node v's representation as

$$h_v = \sigma(w_1 x_v + w_2 \cdot \text{mean}_{u \in N(v)}(x_u) + b), \tag{17.5}$$

where $N(v)$ is the set of v's neighbors in the graph.

Consider a four-node layering chain, $A \to B \to C \to D$, extending Chapter 15's shell-company example, with initial suspicion scores $x_A = 0.9$ (a flagged placement account), $x_B = 0.2$, $x_C = 0.1$, and $x_D = 0.05$ (an apparently unremarkable integration account), and neighbor sets

$$N(A) = \{B\},\ N(B) = \{A, C\},\ N(C) = \{B, D\},\ N(D) = \{C\}.$$

With weights $w_1 = 0.6$, $w_2 = 0.8$, $b = -0.05$, one round of mean-aggregation gives

$$h_B = \sigma\big(0.6 \times 0.2 + 0.8 \times \tfrac{0.9+0.1}{2} - 0.05\big) = \sigma(0.47) = 0.6154$$

and

$$h_C = \sigma\big(0.6 \times 0.1 + 0.8 \times \tfrac{0.2+0.05}{2} - 0.05\big) = \sigma(0.11) = 0.5275,$$

compared to $h_A = \sigma(0.65) = 0.6570$ and $h_D = \sigma(0.06) = 0.5150$. Node B's updated score, 0.615, is far above its raw feature of 0.2 because it is one hop from the flagged account A, while node C, two hops away, reaches only 0.528, barely above D's 0.515: after one round, A's signal has not yet arrived there, and a second round is needed to carry it that far. A model that scored each account independently, as Chapter 15's original logistic regression did, would miss this network-propagated risk.

Multi-Modal Fusion via Cross-Attention

A single financial decision often draws on several modalities at once: an image feature from a CNN, a sentiment embedding from Chapter 16's text pipeline, and a tabular summary like Section 17.2.4's regression inputs. Combining them well matters as much as extracting each. Cross-attention, a direct extension of Chapter 16's self-attention (Section 16.4.1), treats one modality's representation as the query and the others as key-value pairs to attend over:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V,$$

as before, but fusing modalities instead of word tokens.

Suppose a tabular query vector $Q = (0.6, 0.8)$ (summarizing Cascade's alternative-data growth direction) attends over an image modality with key $K_{\text{img}} = (0.9, 0.1)$ and value $V_{\text{img}} = (0.7, 0.2)$, and a text-sentiment modality with key $K_{\text{txt}} = (0.2, 0.9)$ and value $V_{\text{txt}} = (-0.3, 0.6)$ (a mildly cautious sentiment reading). With $d_k = 2$, the scaled dot-product scores are

$$Q \cdot K_{\text{img}} / \sqrt{2} = 0.62 / 1.4142 = 0.4384$$

and

$$Q \cdot K_{\text{txt}} / \sqrt{2} = 0.84 / 1.4142 = 0.5940;$$

applying softmax gives attention weights 0.4612 on the image modality and 0.5388 on the text modality. The fused representation is

$$0.4612(0.7, 0.2) + 0.5388(-0.3, 0.6) = (0.161, 0.416),$$

a single vector that leans slightly toward the cautious text signal, and that a downstream model (a trading rule like Section 17.2.4's, a credit decision, a fraud flag) can consume in place of either modality, without the analyst hand-specifying how much weight each deserves.

Wulin.Suo@Queensu.ca

Self-Supervised Contrastive Pretraining

Section 17.5.2's transfer-learning example assumed a pretrained backbone was available. Contrastive pretraining is one common way to train such a backbone without any human labels, which matters for satellite imagery, where labeled examples (this patch contains three cars) are far scarcer than raw imagery. The idea is to take two randomly augmented views of the same image, a small crop and rotation, say, and train the encoder to pull their embeddings together (a positive pair) and push a different image's embedding away (a negative pair), using only same-versus-different as the training signal.

Suppose an encoder produces embedding $a = (0.6, 0.8)$ for one crop of a satellite patch, $p = (0.55, 0.83)$ for a second, differently augmented crop of the *same* patch, and $n = (0.1, 0.9)$ for a crop of a different patch. Using Chapter 16's cosine similarity (Section 16.3.3), $\cos(a, p) = 0.9983$ and $\cos(a, n) = 0.8614$: two views of the same scene are closer than a different scene, as intended.

A contrastive loss converts these similarities into a probability that the positive pair is correctly identified among the candidates,

$$\Pr(\text{positive}) = \frac{e^{\cos(a,p)/\tau}}{e^{\cos(a,p)/\tau} + e^{\cos(a,n)/\tau}}, \tag{17.6}$$

a softmax over similarities analogous to the cross-attention softmax above, with a temperature τ controlling how sharply the model is pushed to separate positive from negative. With $\tau = 0.1$, $\Pr(\text{positive}) = e^{9.983}/(e^{9.983} + e^{8.614}) = 0.797$, well above chance (50%) before any training. Training drives this probability toward 1 by adjusting the encoder so that same-image crops end up closer together and different-image crops farther apart.

Trained this way on a large unlabeled archive of satellite imagery, the encoder becomes the frozen backbone that the transfer-learning example fine-tuned with a handful of labeled examples, closing the loop between the chapter's two ways of coping with scarce labels.

17.6 Case Vignette: Satellite Imagery and the Rise of Alternative Data

The parking-lot example this chapter has used throughout is not hypothetical in its origins. In 2009, brothers Tom and Alex Diamond conceived the idea of counting cars in retailer parking lots from satellite imagery to anticipate revenue, and founded RS Metrics to sell the data, starting with McDonald's locations and later covering Lowe's, Home Depot, and dozens of other retailers. The approach reached wider attention in 2010, when UBS analyst Neil Currie bought parking-lot counts for 100 Walmart stores and published them ahead of Walmart's earnings release, an early public demonstration that satellite data could anticipate a reported number. A later academic study using RS Metrics imagery from 2011 to 2017, covering 44 major U.S. retailers including Walmart, Target, Costco, and Whole Foods, confirmed that year-over-year change in car counts predicts quarterly sales, and put the resulting informational edge at 4% to 5% in the three trading days around an earnings announcement, a substantial return for so short a window and a large-scale counterpart to Section 17.2.3's worked example.

A separate study of how that coverage spread across retailers found effects beyond rewarding the subscribing funds: retailers that gained satellite coverage subsequently saw more informed short selling, less-informed individual buying, and lower stock liquidity around their earnings reports. Unequal access to alternative data can widen the information gap between sophisticated and ordinary investors without making prices more accurate any faster, the access-asymmetry concern Section 17.3.3 raises about licensing cost and exclusivity.

Satellite data extends well beyond retail. Orbital Insight built a comparable business in commodities: by measuring the shadows cast by the floating roofs of many crude-oil storage tanks (a floating roof sits on the stored oil, so its height reveals how full the tank is), the firm estimates inventory across more than 27,000 tanks worldwide. During the demand shock of the early 2020s pandemic, this work showed global storage running at roughly 55% of an estimated six-billion-barrel capacity, a data point no government agency could have produced as quickly. Satellite-based financial data services are estimated to generate over $500 million in annual revenue, within an alternative-data market on which Wall Street is estimated to spend roughly $3 billion a year, and a single institutional satellite subscription can run from $50,000 to $500,000 per year, so this edge is available mainly to well-capitalized participants able to pay for exclusive or near-exclusive access.

Wulin.Suo@Queensu.ca

17.7 Challenges in Alternative Data and Deep Learning in Finance

Several challenges recur across this chapter's techniques and connect directly to themes from earlier chapters.

A short history limits statistical confidence. Sections 17.2.3 and 17.2.4 fit regressions on only eight quarters, because that is all the history a new alternative-data feed typically has. Chapter 6's warnings about overfitting with few observations apply with particular force, since a vendor's dataset is often too new to have lived through a full economic cycle.

Non-stationarity compounds with vendor changes. Financial relationships shift as markets evolve (Chapter 6), and alternative data adds a second source of instability: a vendor can silently change its imagery source, computer-vision model, or store coverage, so a model's apparent degradation may reflect a change at the vendor rather than in the financial relationship.

Deep architectures trade interpretability for representational power. A CNN's learned filters and an RNN's hidden state are far harder to interpret than Section 17.2.3's single regression coefficient, a tension Chapter 18 develops, and one that matters more when the input is as opaque to manual review as a stack of satellite pixels.

Integration is usually the hard part, not modeling. Aligning several sources in time and by firm identity, and confirming that each was available before the date being modeled, typically takes more effort than fitting the model, the reverse of the market-data-only settings of Chapters 12 and 13.

Cost and exclusivity concentrate the advantage. Section 17.6's subscription costs mean that alternative data's edge accrues mostly to well-resourced funds able to license several exclusive feeds, an access asymmetry with no counterpart in the public market and fundamental data most of this book uses.

Model risk governance now has to reach the vendor, not just the model. Chapter 15's three-lines-of-defense framework validates a single model's inputs, outputs, and ongoing performance. An alternative-data pipeline adds a layer it must now cover, the vendor relationship, since a model's validation is only as good as the vendor's coverage, delivery timing, and methodology upstream of it, none of which the fund controls or, usually, can fully observe.

17.8 Key Takeaways

Alternative data (satellite imagery, web traffic, card-transaction panels) fills the row Chapter 1's data table reserved for it: non-traditional sources that proxy for economic activity before it is officially reported. A single proxy, satellite car counts, explained 87% of a retailer's quarterly revenue variation in Section 17.2.3's example, and combining several sources produced a useful but imperfect earnings-surprise signal (an 80% hit rate, not 100%) in Chapter 13's evaluation framework. Raw readings typically need seasonal adjustment and peer-relative normalization first (Section 17.3.2).

Extracting signals from raw data, rather than from a vendor's processed number, needs architectures beyond the feedforward network: convolutional networks (Section 17.4) exploit an image's grid through weight-shared filters, recurrent networks exploit a sequence's order through a hidden state, and transformer-based self-attention increasingly unifies both roles. None of this removes alternative data's quality problems (Section 17.3.3): coverage bias, vendor black-box processing, look-ahead bias from delivery lag, and cost-driven alpha decay, which a modeler must treat as seriously as the modeling itself.

Respecting delivery lag changes which trades a model recommends, not merely how good its backtest looks (Section 17.3.1), and an autoencoder can turn the same toolkit on the data itself to flag a quarter worth double-checking. Rather than buying a vendor's ESG score, whose disagreement with other providers Chapter 12 documented at 38% to 71% pairwise correlation, a fund can build a narrow, auditable environmental-tone signal from a company's own disclosures (Section 17.5.1), at the cost of adding one more differently scoped measurement to the field.

Section 17.5.2 went deeper: transfer learning fine-tunes a CNN's frozen features on a small sample, contrastive pretraining trains that backbone with no labels at all, graph neural networks let risk propagate across related entities that Chapter 15's layering chain could only describe, and cross-attention fuses image, text, and tabular modalities into one representation without hand-specified weights.

Wulin.Suo@Queensu.ca

17.9 Suggested Exercises

17.1 Using Section 17.2.3's method, refit the regression after adding a ninth quarter with car-count growth of -2.0% and revenue growth of -1.5%, and report the new slope, intercept, and R^2.

17.2 Explain, using Table 17.2, why an R^2 of 0.870 from a single alternative-data proxy is considered strong in this context, contrasting it with Chapter 6's discussion of low signal-to-noise ratios in financial prediction generally.

17.3 Using Section 17.2.4's fitted coefficients, compute the predicted surprise for a tenth quarter with car, web, and card growth of -2.0%, -1.0%, and -1.5% respectively, and state whether the resulting signal is long or flat.

17.4 Explain, using Q5 in Table 17.3, why a strongly positive fitted surprise (2.46%) can still be followed by a negative next-day return (-0.4%), and describe one piece of earnings-day information the alternative-data model in this chapter cannot capture.

17.5 Using Section 17.4's convolution example, recompute the 3×3 feature map using a 2×2 filter of all 0.5s instead of all 0.25s, and explain how the result relates to the original feature map.

17.6 Using the same convolution example, apply 2×2 max pooling (taking the maximum of each non-overlapping 2×2 block) to the original 3×3 feature map's top-left 2×2 block, and state the resulting pooled value.

17.7 Using Section 17.4's RNN example, recompute h_1, h_2, and h_3 if w_h is increased from 0.4 to 0.8 (holding w_x, b, and the inputs fixed), and explain in words how a larger w_h changes how much the hidden state is influenced by earlier time steps versus the current input.

17.8 Explain why a convolutional neural network's weight sharing makes it far more practical than a plain feedforward network for a 1000×1000-pixel satellite image, referencing the parameter-count argument in Section 17.4.

17.9 Using this chapter's case vignette, explain why the 2010 UBS Walmart parking-lot report is a natural precedent for Section 17.2.3's worked example, and describe why the same informational edge tends to shrink as more funds license the same data source.

17.10 Pick two of the data-quality issues in Section 17.3.3 and describe a concrete scenario, not already used in the text, in which that issue could cause an alternative-data-based trading or credit model to fail despite technically correct code.

17.11 Compare this chapter's satellite car-count regression (Section 17.2.3) to Chapter 5's binomial and Black-Scholes-Merton option-pricing models in terms of what each treats as the source of uncertainty, and explain why alternative-data models are typically evaluated by out-of-sample predictive accuracy rather than by no-arbitrage argument.

17.12 Using Section 17.3.1's method, explain why the realistic, lagged-card model's higher R^2 (0.873 versus 0.837) does not mean it is simply the better trading model, referencing the difference in which quarters each version actually signals.

17.13 Using Section 17.3.2's method, recompute the peer-adjusted regression's slope and R^2 if the sector-average car-count growth for Q5 is revised from 2.5% to 5.0% (a sector-wide surge that quarter), holding all other quarters fixed, and explain why the peer-adjusted model's R^2 falls even though Cascade's own reported revenue growth has not changed.

17.14 A vendor offers a trial dataset covering only the three quarters (of Table 17.3) in which its data predicted the following quarter's earnings surprise correctly. Using Section 17.2.2's discussion, explain what is wrong with evaluating the vendor's data quality using only this trial.

Wulin.Suo@Queensu.ca

17.15 Using Section 17.2.2's cost-benefit method, recompute the expected net annual benefit if the fund can only deploy a $20 million position per signaled event (instead of $50 million), holding the signal frequency, edge, and data costs fixed, and state whether the subscription is still worth its $400,000 annual cost.

17.16 Using Section 17.4's autoencoder example, compute the reconstruction error and relative error for Q2 (car $= -1.5$, web $= -0.5$, card $= -1.0$) using the same encoder/decoder weights $w = (0.5, 0.3, 0.2)$, and state whether Q2 would be flagged ahead of Q7 (relative error 39.4%) for data-quality review.

17.17 Using Section 17.5.2's transfer-learning example, perform a second gradient-descent step starting from the updated weights $w = (0.549, 0.464)$, $b = 0.113$, and report the new prediction, confirming that it continues to move toward the true label of 1.

17.18 Using Section 17.5.2's graph neural network example, compute h_A and h_D's updated scores using the same weights, and explain, using the network's chain structure $A \to B \to C \to D$, why h_D's score barely moves from its raw feature while h_B's moves substantially.

17.19 Using Section 17.5.2's cross-attention example, recompute the fused output if the text modality's key becomes $K_{\text{txt}} = (0.8, 0.2)$ (a much less distinctive key vector, more similar to K_{img}), and explain in words why the two attention weights move closer to 50/50.

17.20 Explain why a graph neural network's mean-aggregation step (Section 17.5.2) is a more direct computational analogue of "guilt by association" than Chapter 15's original, per-account fraud-scoring logistic regression.

17.21 Using Section 17.5.2's contrastive-pretraining example, recompute Pr(positive) if the temperature τ is raised from 0.1 to 1.0 (holding the two cosine similarities fixed), and explain in words why a higher temperature makes the model's implied preference for the true positive pair less extreme.

17.22 Using Section 17.4's LSTM worked example, recompute the forget gate f_1 and the resulting fraction of c_1 surviving to c_3 (that is, $f_2 \times f_3$) if the forget-gate bias b_f were 0.5 instead of 2.0, holding every other weight fixed, and explain why a smaller bias makes the LSTM behave more like the plain RNN example earlier in this section.

17.23 Explain why the plain RNN's memory-decay problem is a consequence of its hidden state being fully overwritten by a multiplicative recursion at every step, while the LSTM's cell state update is additive, and describe what would happen to the LSTM's own memory retention if its forget gate were trained to a value close to 0 instead of close to 1.

17.24 Using Section 17.5.1's method, recompute the tone score for a fifth sentence, "The board remains confident that emissions will decline further even as regulatory headwinds intensify," using the same positive and negative word lists, and explain why this sentence's mixed vocabulary makes its tone score a poor summary of whether the sentence is good or bad news.

17.25 Explain why Section 17.5.1's in-house tone score and Chapter 12's vendor-style ESG scores are not directly comparable even though both are commonly described as "an ESG measurement," and describe which one of Berg, Kölbel, and Rigobon's three sources of rating divergence, scope, measurement, or weight, this incomparability illustrates.

17.26 The training-discipline passage at the end of Section 17.4.4 reports mean walk-forward test RMSEs of 1.128 (naive), 1.074 (AR(2)), 1.066 (gradient boosting), and 1.050 (LSTM) against a noise floor of 1.037. (a) Compute each model's excess RMSE over the noise floor, and the percentage of the naive forecast's excess that each of the other three models removes. (b) Explain why the LSTM's 2.3% improvement over the AR(2) and its share of the removable error tell different stories, and which of the two a model-risk reviewer should ask to see. (c) Explain why reshuffling the 1,500 observations into five random folds, instead of walk-forward blocks, would be expected to flatter the two flexible models, the LSTM and gradient boosting, more than the naive forecast and the AR(2), given that each input is a window of the ten preceding values.

Wulin.Suo@Queensu.ca

17.27 A vendor's foot-traffic series for a chain of garden centers has predicted the chain's quarterly revenue growth well over five years. Using the "Is this signal causal?" paragraph of Section 17.3.3, describe one mechanism through which the series measures revenue directly and one confounder that could produce the same association without it, and propose a check on the data that would help tell the two apart. Then explain why the chain's own management, deciding whether a promotion that raises foot traffic will raise revenue, needs a different kind of evidence from a fund trading the stock.

17.28 **Challenge (real data).** Download at least two years of weekly Google Trends search-interest data (via `pytrends` or manually from trends.google.com, no API key required) for a public retailer's brand name, and quarterly reported revenue for the same company from the SEC EDGAR XBRL `companyfacts` API (as in Chapter 4's challenge exercise). (a) Following Section 17.2.3's method, regress the company's quarter-over-quarter revenue growth on the average quarter-over-quarter growth in search interest, and report the fitted slope, intercept, and R^2. (b) Using Section 17.3.1's distinction between a realistic, lagged signal and a look-ahead-biased one, identify what reporting lag your search-interest data is actually available at relative to the revenue figure it is meant to predict, and explain whether your regression in (a) would still be usable in real time. (c) Following Section 17.2.2's cost-benefit framework, discuss one alternative-data quality issue from Section 17.3.3 other than look-ahead bias that is specific to Google Trends data, such as index normalization or query ambiguity, and explain how it could produce a spuriously strong or weak regression fit.

17.10 Further Reading

- Counts (2019), *How Hedge Funds Use Satellite Images to Beat Wall Street, and Main Street*. A research summary of the RS Metrics parking-lot car-count studies underlying Section 17.2.3 and this chapter's case vignette.
- Katona et al. (2025), "On the Capital Market Consequences of Big Data: Evidence from Outer Space." The academic study of RS Metrics satellite imagery (2011-2017, 44 U.S. retailers) quantifying the earnings-window informational edge discussed in Section 17.6; it also documents the short-selling, liquidity, and information-asymmetry effects of satellite coverage discussed in this chapter's expanded case vignette.
- Feng and Fay (2022), "An Empirical Investigation of Forward-Looking Retailer Performance Using Parking Lot Traffic Data Derived from Satellite Imagery." A complementary study of satellite-derived parking-lot data's power to predict retailer sales, underlying Section 17.2.3's worked example.
- LeCun et al. (2015), "Deep Learning." A survey covering convolutional and recurrent neural network architectures underlying Section 17.4.
- Goodfellow et al. (2016), *Deep Learning*. A comprehensive textbook treatment of convolution, recurrence, and the attention mechanisms Chapter 16 and Section 17.4 both draw on.
- Grinsztajn et al. (2022), "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" The benchmark of tree ensembles against deep networks on 45 tabular datasets behind the training-discipline passage in Section 17.4.4, with an analysis of why trees keep winning on that kind of data.
- Hochreiter and Schmidhuber (1997), "Long Short-Term Memory." The original paper introducing the LSTM architecture mentioned as an extension of Section 17.4's plain recurrent network.
- Dosovitskiy et al. (2021), "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." The original vision transformer paper underlying Section 17.4's discussion of transformers as a unifying architecture.
- Kolanovic and Krishnamachari (2017), *Big Data and AI Strategies: Machine Learning and Alternative Data Approach to Investing*. An industry survey of alternative data sources and their use in systematic investing, covering the categories in Table 17.1.

Wulin.Suo@Queensu.ca

- Berg et al. (2022), "Aggregate Confusion: The Divergence of ESG Ratings." The empirical study of cross-provider ESG rating disagreement discussed in Chapter 12 and revisited in Section 17.5.1 from the standpoint of building an in-house text-derived signal rather than buying a vendor's score.
- Hinton and Salakhutdinov (2006), "Reducing the Dimensionality of Data with Neural Networks." A foundational paper on autoencoders, underlying Section 17.4's reconstruction-error anomaly-detection example.
- Pan and Yang (2010), "A Survey on Transfer Learning." A comprehensive survey of the pretrained-backbone fine-tuning approach worked through in Section 17.5.2.
- Kipf and Welling (2017), "Semi-Supervised Classification with Graph Convolutional Networks." A foundational paper on the graph neural network mean-aggregation update used in Section 17.5.2's shell-company network example.
- Chen et al. (2020), "A Simple Framework for Contrastive Learning of Visual Representations." The SimCLR paper underlying Section 17.5.2's contrastive-pretraining worked example.
- He et al. (2016), "Deep Residual Learning for Image Recognition." The original ResNet paper underlying Section 17.4's discussion of shortcut connections and the vanishing-gradient problem in deep CNNs.

Wulin.Suo@Queensu.ca